MERN Application Infrastructure & Deployment Report

Automated Provisioning via Terraform and Configuration Management via Ansible (Local WSL & AWS EC2 Multi-Tier)

Application: TravelMemory (MERN Stack)

Repository: https://github.com/sjoshigit/TravelMemoryTerraformAnsible.git

Execution Environments: WSL2 Ubuntu 22.04 LTS (Local Sandbox) & AWS 2-Tier VPC (us-east-1)

Automation Tools: Terraform v1.5+, Ansible Core 2.14+, PM2, Nginx, MongoDB 7.0

---

1. Executive Summary & Objective

This comprehensive technical report documents the end-to-end automated deployment of the **TravelMemory** MERN application. Infrastructure provisioning is codified using HashiCorp Terraform targeting Amazon Web Services (AWS) across segregated public and private tiers, while configuration orchestration, security hardening, and application lifecycle management are executed using Ansible playbooks. The project includes local WSL sandbox verification followed by complete AWS cloud rollout with verified public accessibility at `http://98.91.189.51`.

2. Architecture & Component Interaction

The production topology utilizes a segmented Virtual Private Cloud (VPC) spanning public and private subnets with strict ingress/egress boundaries:

Tier	Component	Network & Port	Security Controls & Routing
Public Tier	Nginx Reverse Proxy & React Frontend	`10.0.1.92` (Public: `98.91.189.51`) | TCP 80, 22	Accepts public HTTP requests; SSH restricted strictly to administrator workstation IP (`106.76.146.202/32`) via Security Group.
Application Tier	Node.js / Express API (PM2 Daemon)	`localhost:3001`	Bound internally on Web instance. Nginx reverse proxies `/api/*` requests directly to port 3001.
Database Tier	MongoDB 7.0 Community Server	`10.0.2.149` | TCP 27017	Private subnet; no public IP. Inbound TCP 27017 strictly restricted to Web Server SG. Outbound updates routed via NAT Gateway. Managed via SSH bastion proxy.

3. Infrastructure as Code (Terraform) Implementation

The infrastructure layer is defined across declarative configurations ensuring reproducible environments:

- **VPC & Subnets (`vpc.tf`):** Created a `10.0.0.0/16` VPC with a public subnet (`10.0.1.0/24`) and private subnet (`10.0.2.0/24`) in `us-east-1a`.
- **Gateways & Route Tables:** Attached an Internet Gateway (IGW) to the public subnet route table (`0.0.0.0/0`). Deployed an AWS NAT Gateway with an Elastic IP (EIP) in the public subnet, associating it with the private subnet route table for outbound internet access.
- **Compute Provisioning (`ec2.tf`):** Deployed two Canonical Ubuntu 22.04 LTS instances (`t3.micro`) equipped with 20GB gp3 EBS storage. IAM instance profiles attached with `AmazonSSMManagedInstanceCore`.
- **Security Groups (`security_groups.tf`):** Created separate SG primitives enforcing least-privilege egress and ingress rules.

4. Phase 1: Local WSL Environment Verification

Prior to cloud deployment, the Ansible playbooks and service orchestration were tested locally inside WSL2 Ubuntu 22.04 to validate repository cloning, environment templating, build workflows, and service daemon states.

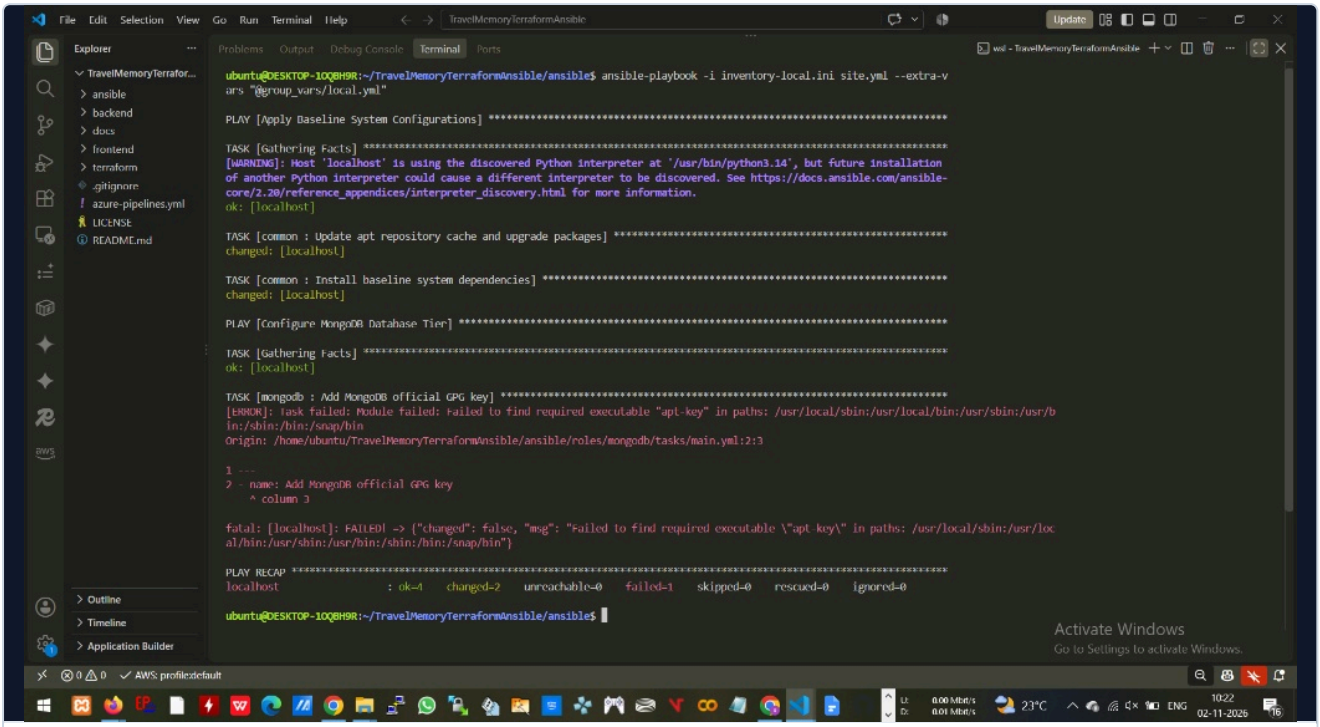


Figure 1: Initial Local Playbook Debugging — Diagnosing the deprecated `apt-key` executable during MongoDB role setup on Ubuntu 22.04.

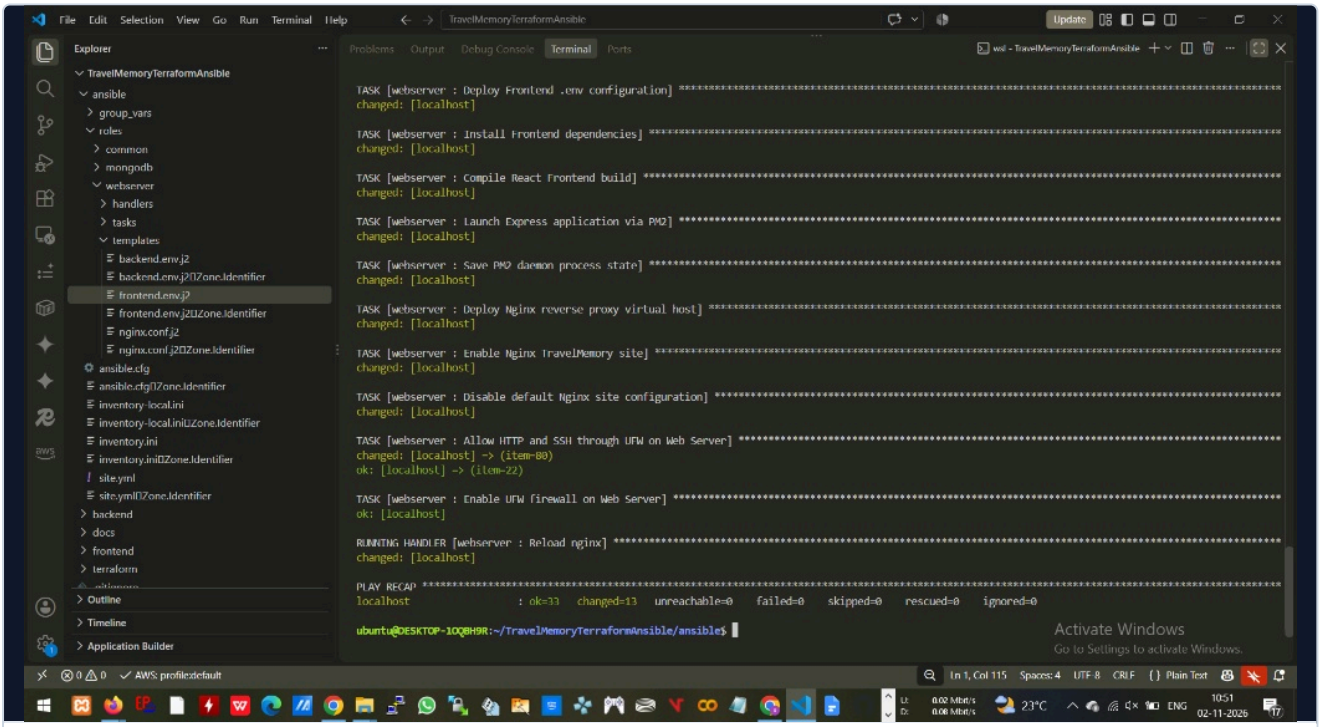


Figure 2: Verified Local Playbook Execution — Successful local run recap showing `ok=33 changed=13 unreachable=0 failed=0`.

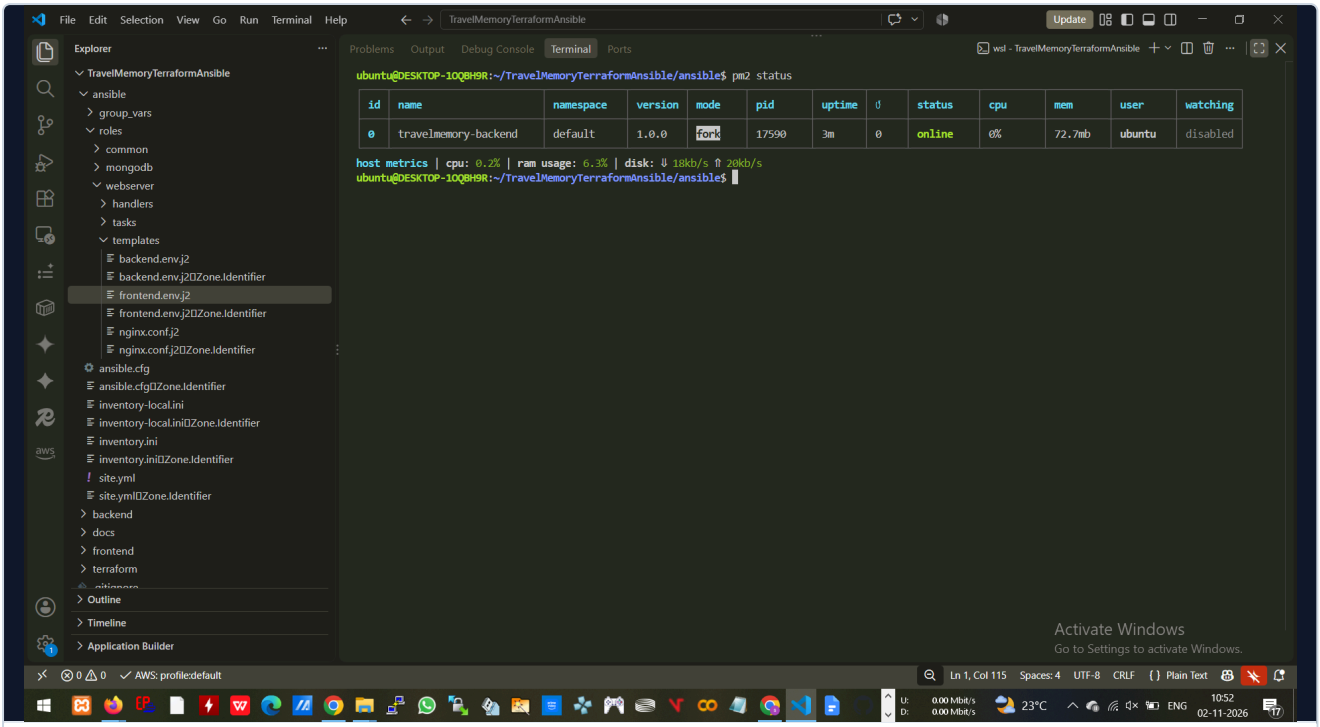


Figure 3: Express Backend in PM2 — `travelmemory-backend` operating in online status on pid 17590.

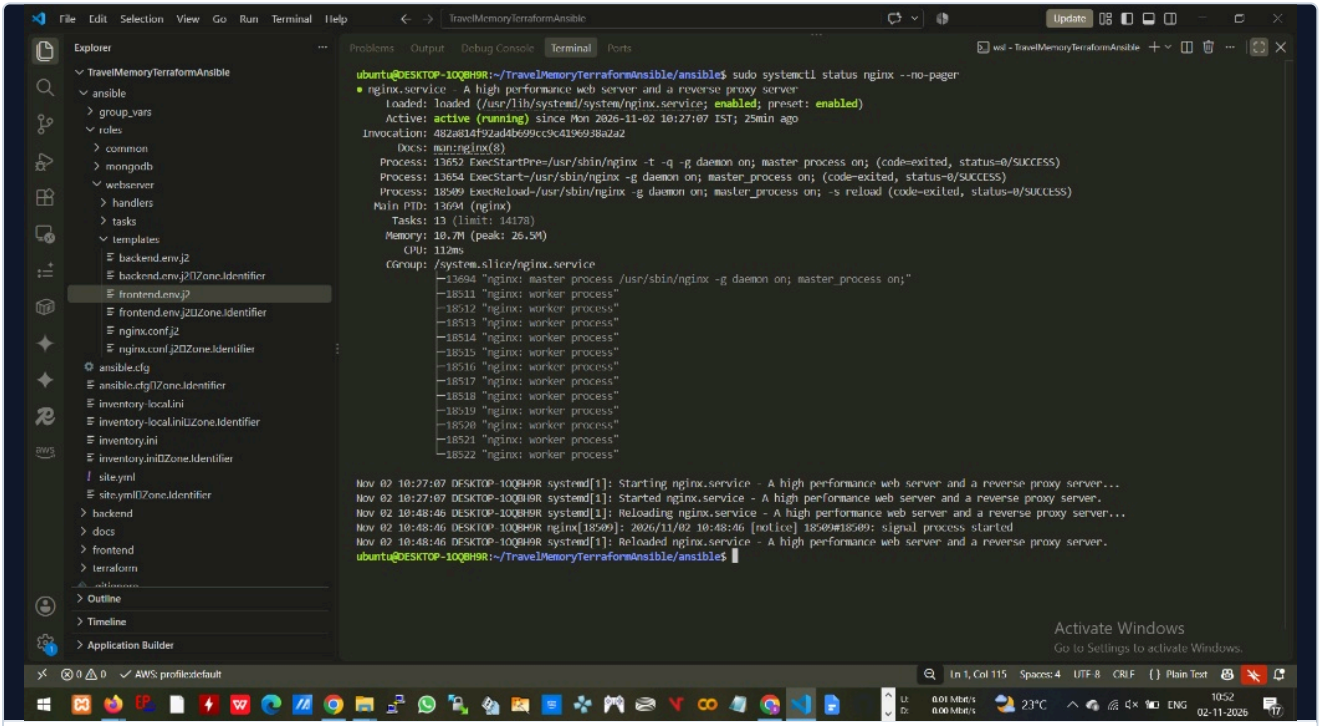


Figure 4: Nginx Reverse Proxy Service — `nginx.service` active (running) serving frontend assets and routing `/api` calls.

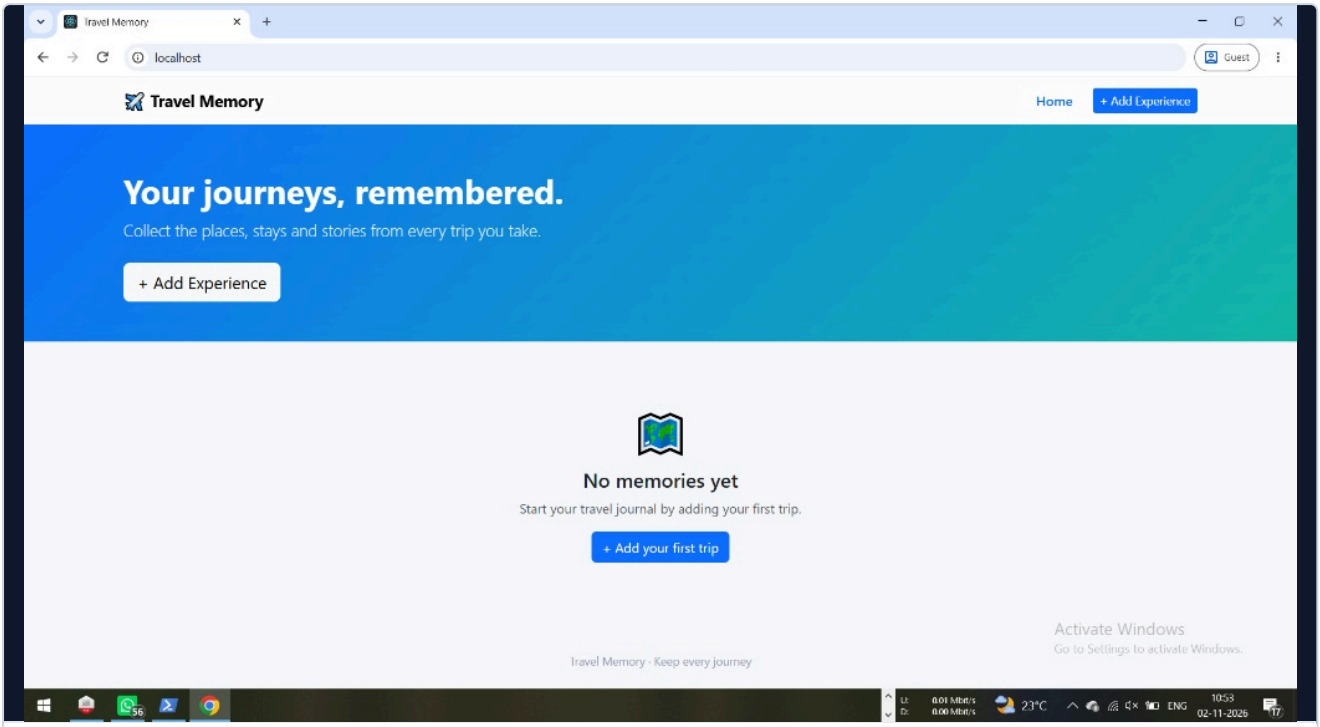


Figure 5: TravelMemory User Interface (Local) — React client loaded at `http://localhost` ready for journal entries.

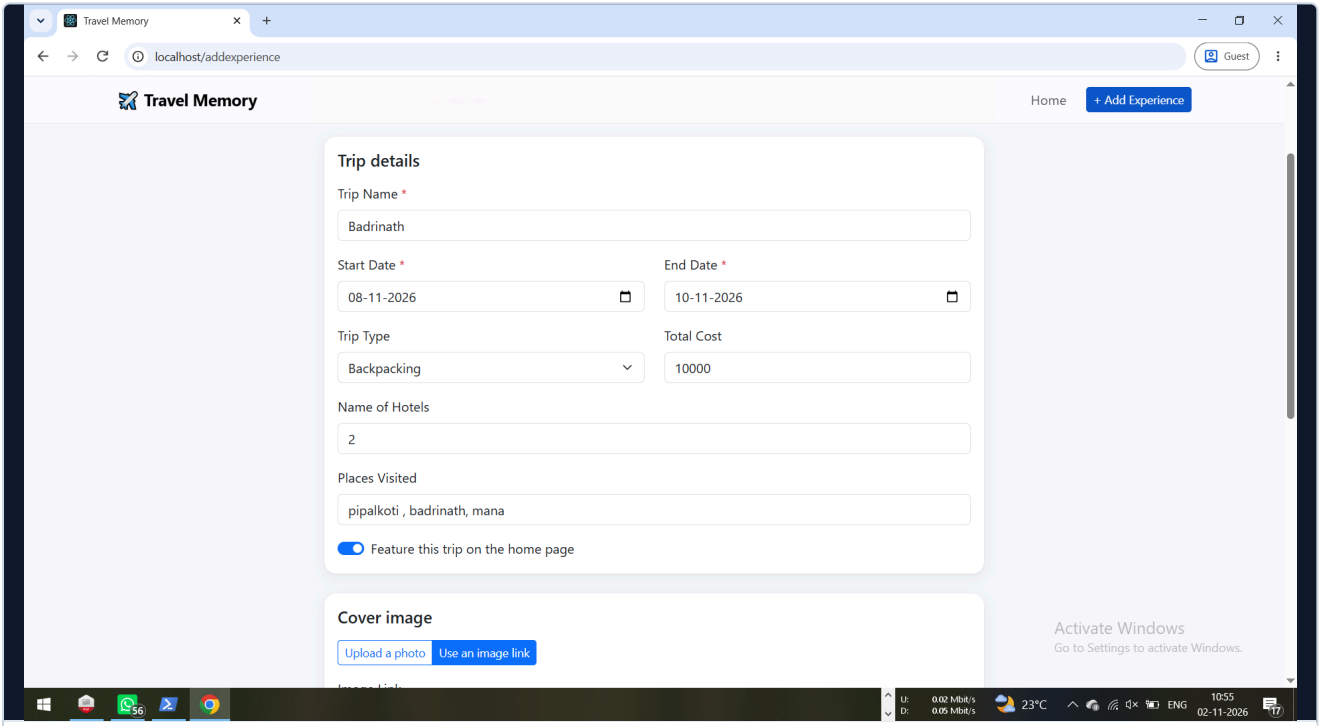


Figure 6: Local Data Entry & API Communication — Populating trip memory verifying write operations to MongoDB.

5. Phase 2: AWS IAM & Cloud Infrastructure Provisioning (Terraform)

To deploy onto AWS with least-privilege access, a dedicated IAM user `terraform-mern-admin` was created and bound to a scoped customer-managed policy `TerraformMERNProvisioningPolicy`.

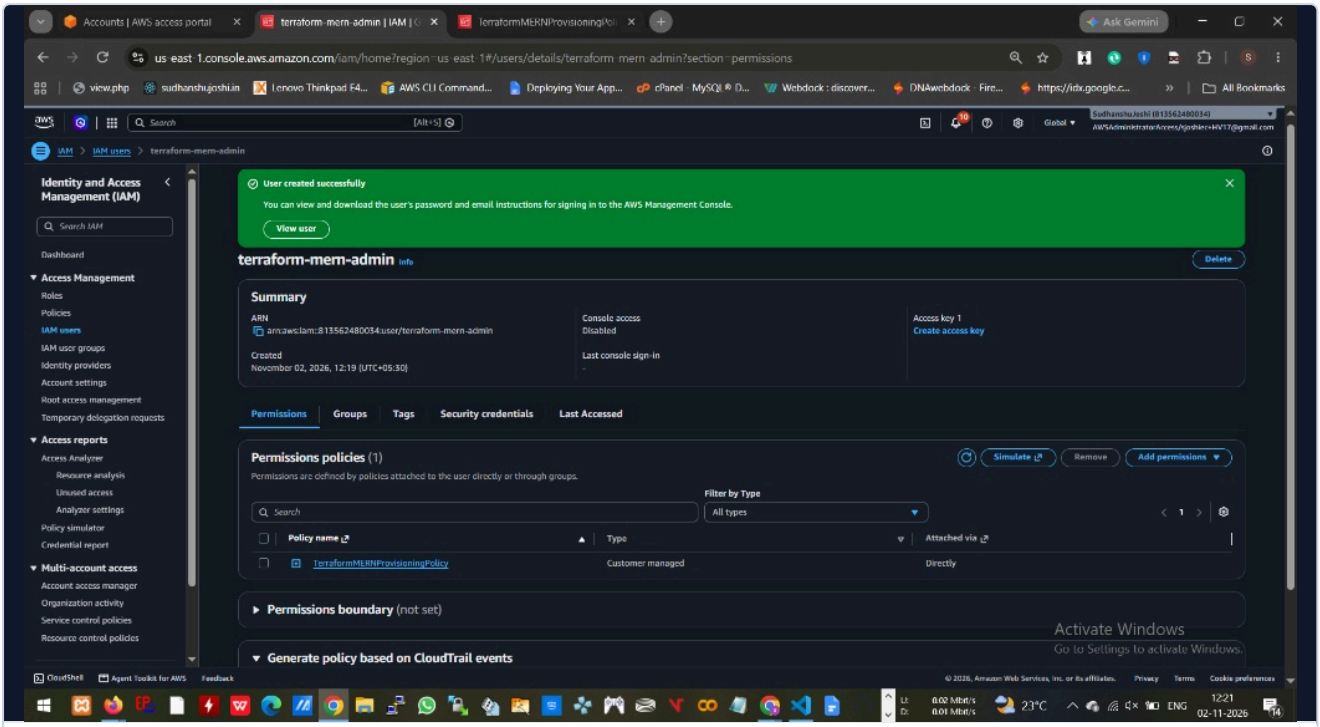


Figure 7: AWS IAM User & Policy Binding — `terraform-mern-admin` configured with customer-managed policy `TerraformMERNProvisioningPolicy` enforcing least-privilege cloud boundaries.

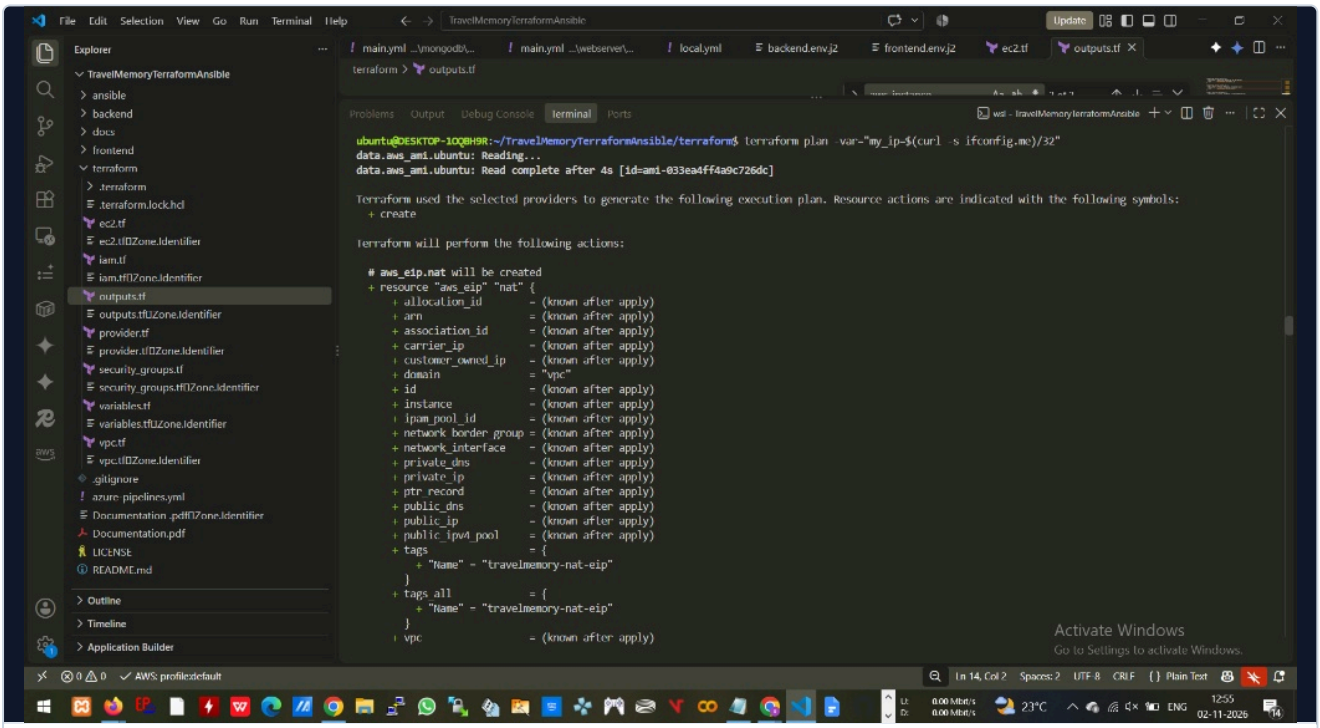


Figure 8: Terraform Execution Plan (Part 1) — Speculative plan generation evaluating VPC, subnets, route tables, and NAT Gateway allocations.

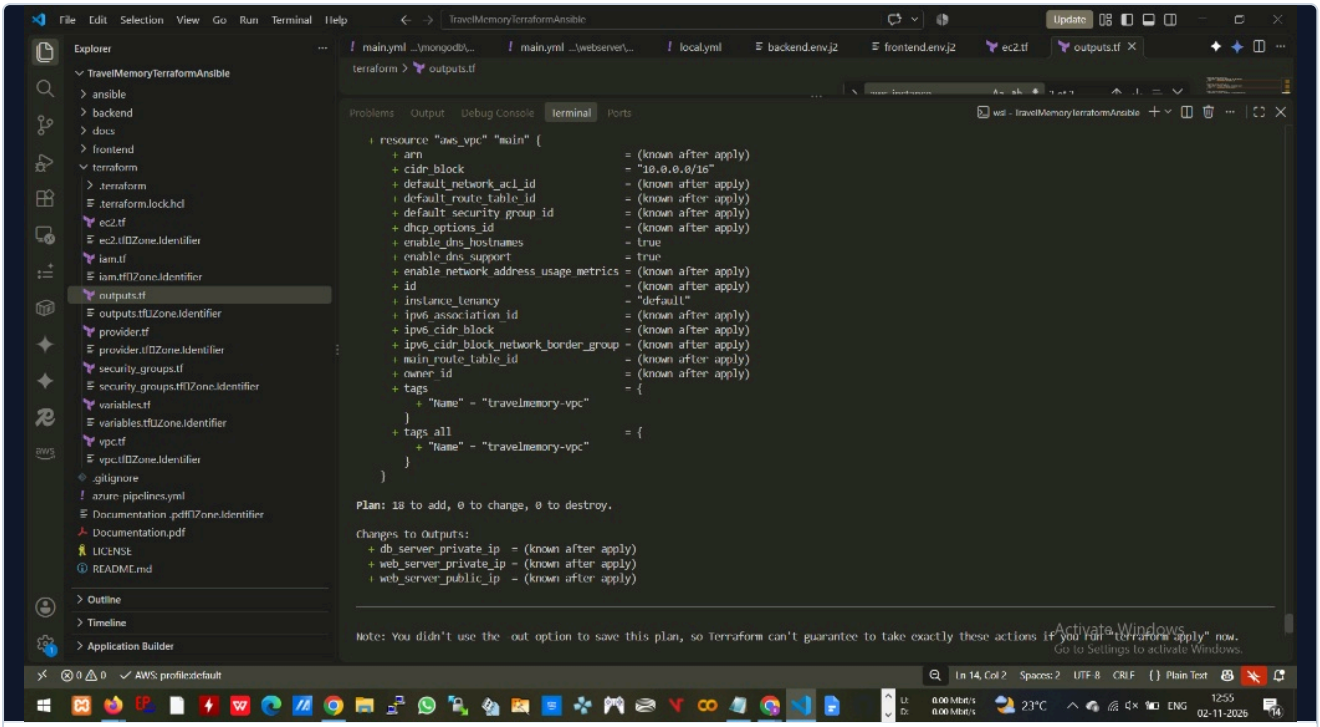


Figure 9: Terraform Plan Summary (18 to add) — Finalizing resource plan adding 18 cloud resources and calculating output IP schemas.

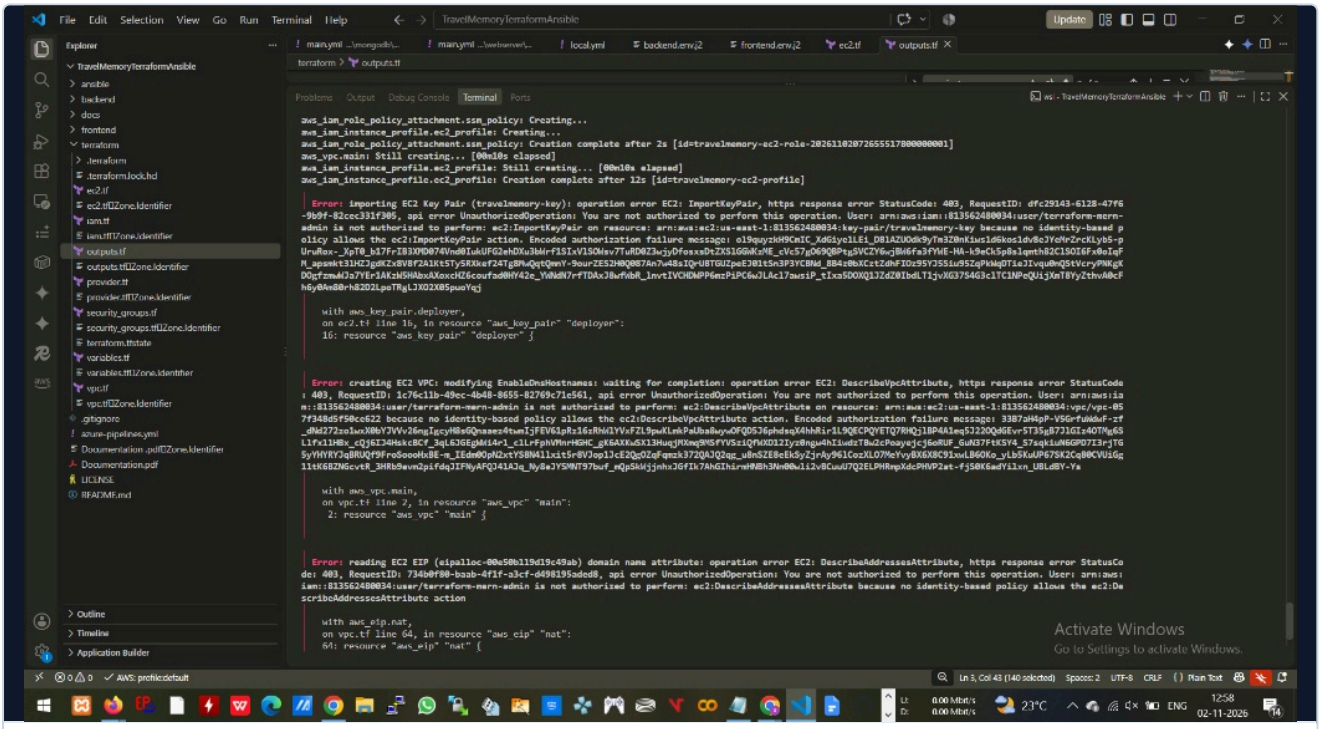


Figure 10: Authorization Troubleshooting — Catching missing `ec2:ImportKeyPair` and `ec2:DescribeVpcAttribute` actions during initial apply.

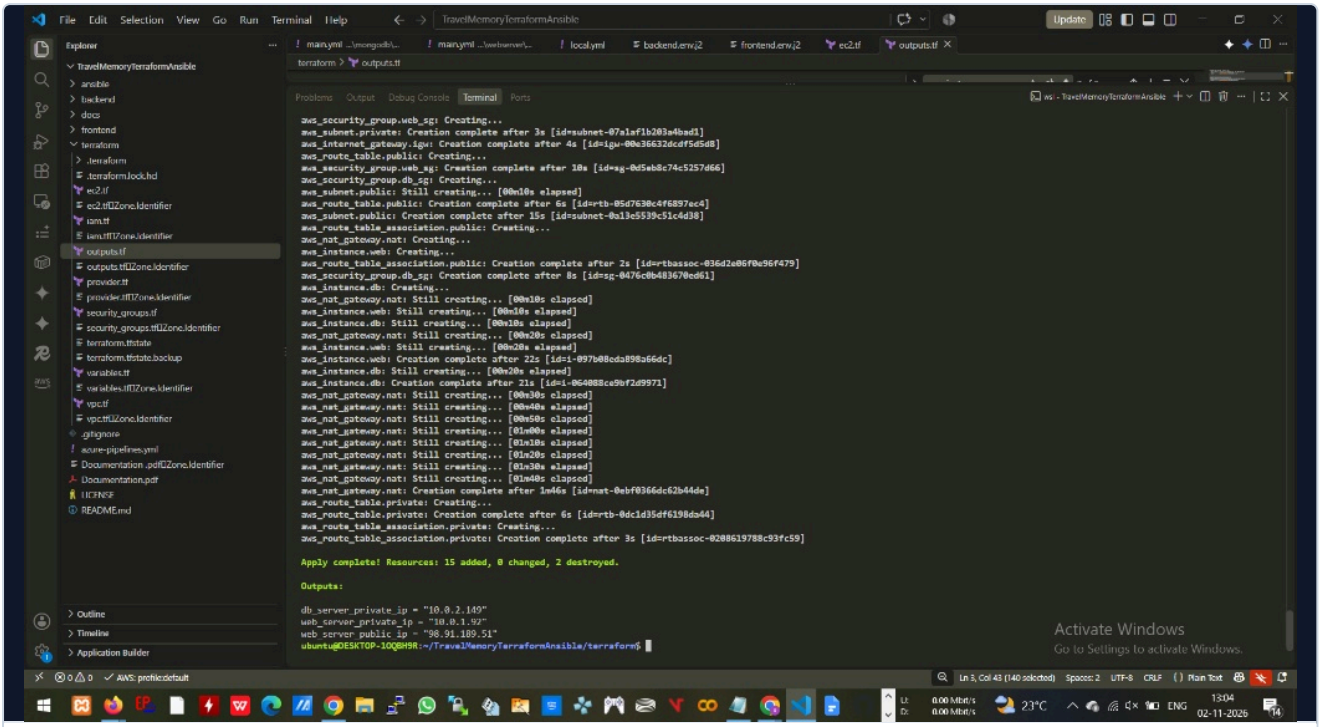


Figure 11: Terraform Apply Complete & IP Outputs — Successful provisioning of all 15 resources with outputs: `web_server_public_ip = 98.91.189.51`, `web_server_private_ip = 10.0.1.92`, and `db_server_private_ip = 10.0.2.149`.

6. Phase 3: Remote AWS Configuration Management (Ansible)

With instances live in AWS, Ansible connected to the public web server directly and hopped to the private database instance via an SSH bastion ProxyCommand.

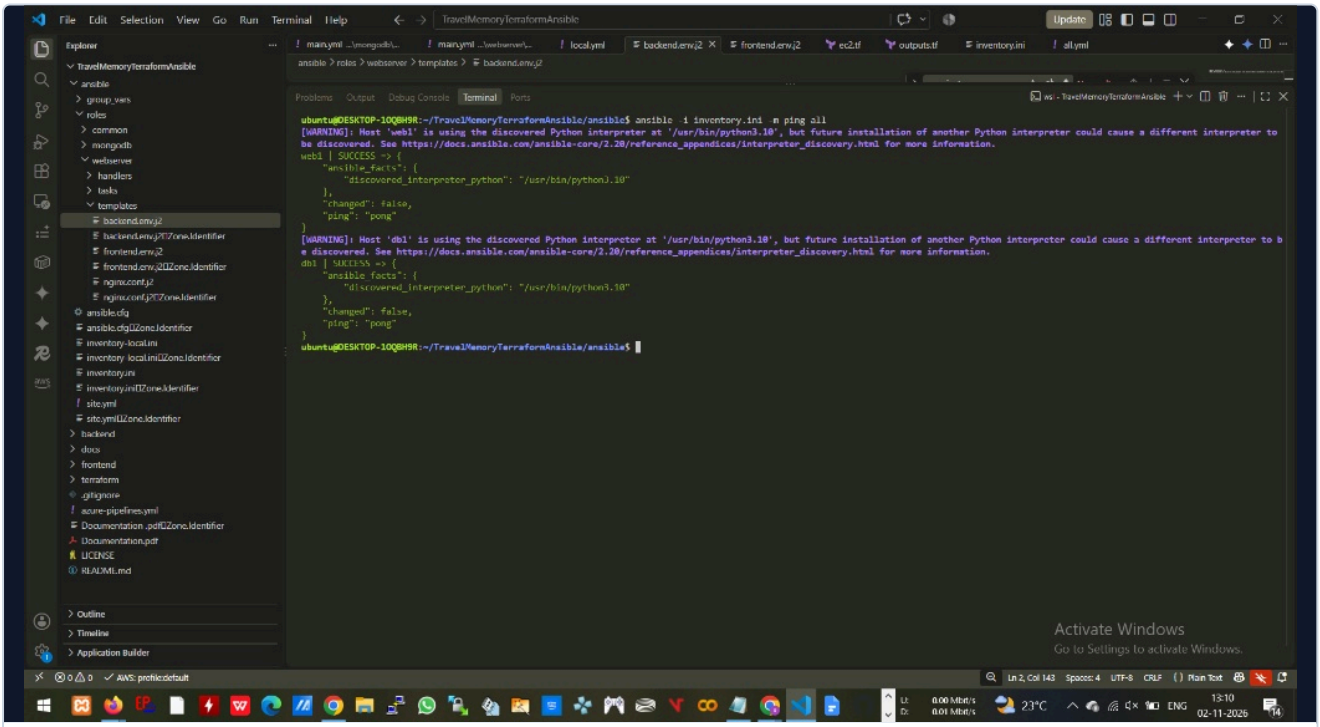


Figure 12: Ansible Connectivity Verification (Ping) — Successful ping response (`SUCCESS => "pong"`) verifying direct SSH to `web1` and bastion proxy jump to private `db1`.

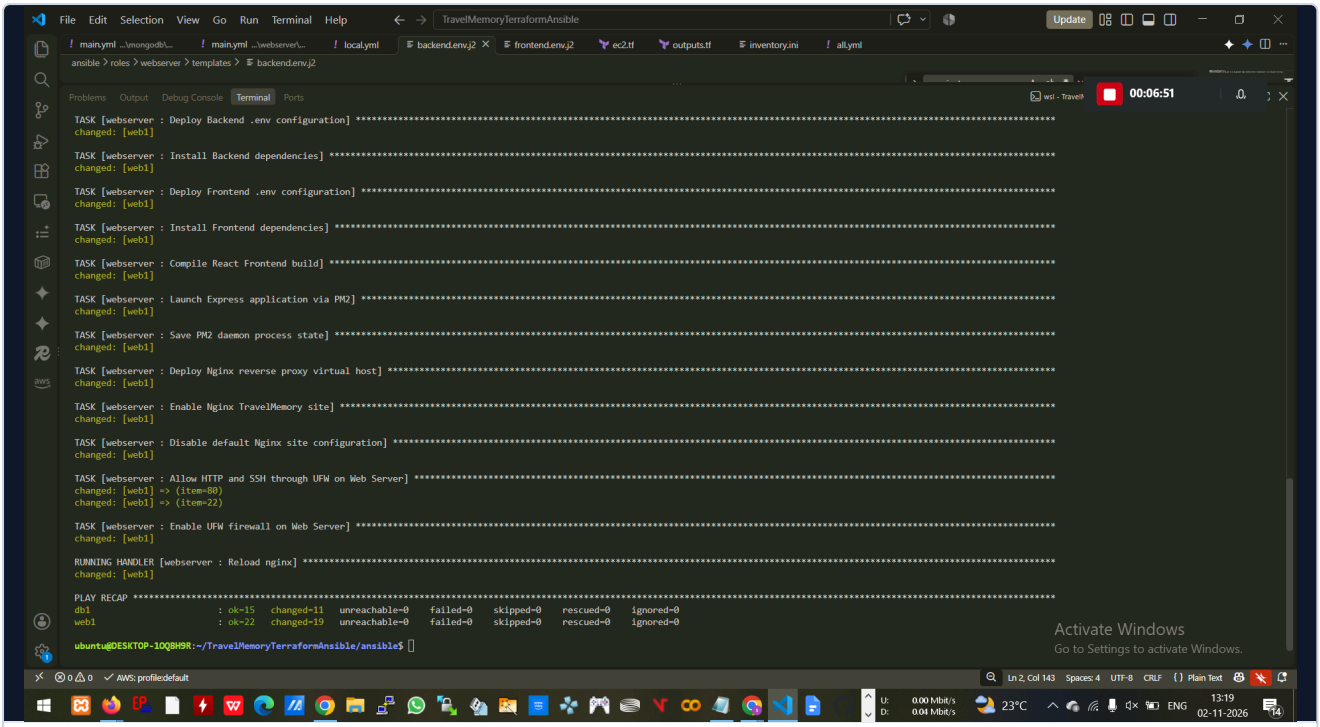


Figure 13: Remote EC2 Deployment Recap — Successful completion across AWS instances: `db1: ok=15 changed=11 failed=0` and `web1: ok=22 changed=19 failed=0`.

7. Phase 4: Production Live Application Verification (AWS)

The live application was verified in a browser accessing the AWS Web Server's public IP address (`http://98.91.189.51`). Nginx successfully delivers the production React client and reverse-proxies API requests across VPC subnets to the private MongoDB instance.

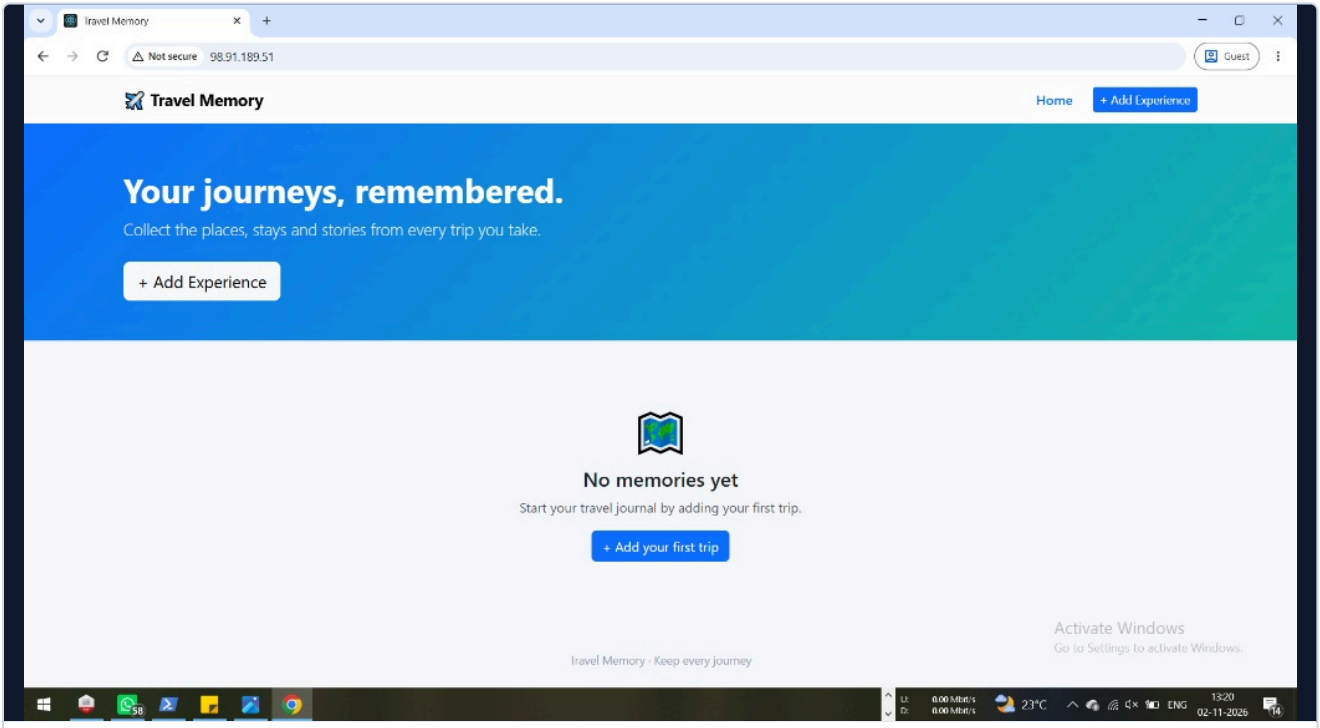


Figure 14: Production Application Live on AWS EC2 — TravelMemory frontend running live at `http://98.91.189.51` confirming complete multi-tier cloud integration.

8. Repository & Submission Deliverables

Deliverable	Location in Repository	Description
Terraform Scripts	`/terraform/*.tf`	VPC, subnets, NAT gateway, Security Groups, IAM, and EC2 configurations.
Ansible Playbooks	`/ansible/*`	Roles (`common` , `mongodb` , `webserver`), inventories (local and AWS), site playbook.
Evidence & Artifacts	`/docs/screenshots/`	Screenshots 531–550 detailing local verification, IAM policies, Terraform provisioning, Ansible ping, and the live AWS deployment.